

SafeVault - Phase 2 Sprint 1 Deliverable

Repositório: desofs2026_thu_ffs_3

Turma: thu_ffs

Sprint: Phase 2 - Sprint 1

Data: 2026-05-18

1. Objetivo da Sprint

Implementar e automatizar o conjunto de práticas de desenvolvimento e testes de segurança exigidas na rubrica 6.2 (Development, Build and Test, Pipeline automation, ASVS), mantendo coerência entre documentação e desenvolvimento.

2. Resumo do que foi feito

- Pipeline CI com build, testes e SCA.
 - SAST via CodeQL.
 - DAST baseline via OWASP ZAP (workflow manual para alvo externo).
 - IAST leve com middleware de detecção de padrões suspeitos.
 - Hardening técnico: segredos removidos do repositório, CSRF dinâmico, erros sem detalhe interno, validação de magic bytes.
 - Documentação completa e rastreabilidade atualizada.
-

3. Desenvolvimento (30%)

3.1 Boas práticas e code review

- Code review formalizado com CODEOWNERS e template de PR:
 - [.github/CODEOWNERS](#)
 - [.github/pull_request_template.md](#)
- Política de revisão: pelo menos 1 revisor e checklist de segurança no PR.

3.2 Hardening técnico implementado

- Segredos removidos de ficheiros versionados:
 - [src/InterfaceAdapters/appsettings.json](#)
 - [src/InterfaceAdapters/appsettings.Development.json](#)
- Validação de segredo e connection string em runtime:
 - [src/InterfaceAdapters/Program.cs](#)
- CSRF token dinâmico com validade de 30 minutos:
 - [src/Infrastructure/Security/CsrfTokenService.cs](#)
 - [src/InterfaceAdapters/Middleware/CsrfTokenMiddleware.cs](#)

- [src/InterfaceAdapters/Controllers/AuthController.cs](#)
- Erros sem detalhe interno, com correlation id:
 - [src/InterfaceAdapters/Middleware/ExceptionHandlingMiddleware.cs](#)
- Validação de magic bytes para uploads:
 - [src/Application/Services/DocumentService.cs](#)

3.3 IAST leve (runtime detection)

- Middleware de detecção de padrões suspeitos (SQLi/XSS/path traversal) em runtime:
 - [src/InterfaceAdapters/Middleware/lastMonitoringMiddleware.cs](#)
 - Controlado por configuração:
 - [src/InterfaceAdapters/appsettings.json](#)
 - [src/InterfaceAdapters/appsettings.Development.json](#)
-

4. Build and Test (30%)

4.1 Execução automatizada de testes

- Pipeline executa build e testes com cobertura:
 - [.github/workflows/ci.yml](#)
- Testes existentes:
 - [tests/ApplicationTests](#)
 - [tests/InfrastructureTests](#)
 - [tests/InterfaceAdaptersTests](#)

4.2 Validação de configuração

- Falha rápida se secrets/connection string estiverem ausentes:
 - [src/InterfaceAdapters/Program.cs](#)

4.3 Análise dinâmica (DAST)

- OWASP ZAP baseline via workflow manual para alvo externo (staging):
 - [.github/workflows/dast.yml](#)
 - [.zap/rules.tsv](#)
-

5. Pipeline automation (20%)

5.1 CI principal

- Build, testes e SCA automatizados em cada push/PR:
 - [.github/workflows/ci.yml](#)

5.2 SAST

- CodeQL configurado e automatizado:
 - [.github/workflows/codeql.yml](#)

5.3 SCA

- dotnet list package --vulnerable automatizado com artefacto:
 - [.github/workflows/ci.yml](#)

5.4 DAST

- OWASP ZAP baseline para alvo configurável via workflow_dispatch:
 - [.github/workflows/dast.yml](#)
-

6. ASVS (15%)

Checklist atualizado e alinhado com evidências reais:

- [phase2_sprint1_asvs_checklist.md](#)
-

7. Comandos e reprodução local

7.1 Variáveis de ambiente obrigatórias

```
ConnectionStrings__DefaultConnection=Host=localhost;Port=5432;Database=safevault;U
sername=postgres;Password=postgres
Jwt__SigningKey=CHANGE_ME_WITH_AT_LEAST_32_CHARACTERS
Jwt__Issuer=SafeVault
Jwt__Audience=SafeVault.Api
```

7.2 Build e testes

```
dotnet restore
dotnet build --configuration Release
dotnet test --configuration Release
```

7.3 DAST manual (ZAP)

Executar o workflow [dast.yml](#) via GitHub Actions e fornecer um URL de staging válido (HTTPS).

8. Rastreabilidade

- Matriz de rastreabilidade Sprint 1:
 - [phase2_sprint1_traceability_matrix.md](#)
-

9. Lacunas conhecidas e próxima sprint

- Integrar DAST automático contra ambiente de staging com dados reais.

- Estender IAST com instrumentação mais profunda (body inspection controlado e correlação request->sink).
- Completar evidências de execução (links diretos para runs da pipeline).